

Midterm Report

Quantum Routines: Implementation, Verification, and Resource Estimation

Kush Aggarwal - agg1810@uw.edu
Thomas Huang - yshuang@uw.edu
Christopher Sharp - sharpc42@uw.edu
Industry Mentor: Mariia Mykhailova (PsiQuantum)

Table of Contents

- Abstract
- 1. Project Background
- 2. Methods
- 3. Results and Analysis
- 4. Future Work
- 5. Project Schedule
- 6. References

Abstract

As quantum computing matures, basic algorithms such as arithmetic algorithms are needed to begin implementing software while abstracting the underlying hardware implementation. These algorithms include adders and multipliers, and are restricted by the rules of quantum computing to be unitary and reversible. They should also be as efficient as possible, utilizing minimal resources. We have implemented the first of several such algorithms for the PsiQuantum Workbench platform and done preliminary benchmarking of their resource use.

1. Project Background

As the industry of quantum computing continues to develop, as hardware implementations are improved upon and finalized, there is attention being directed towards designing optimized algorithms and implementing classical routines on quantum computers so that companies, institutions, and universities can begin designing software for quantum computing without requiring the hardware to be finalized. This early and active software development will allow the goal of quantum computing to complement and improve on classical computing to be better realized.

To do this, we must reformulate classical computing routines into quantum-friendly forms. In particular, the rules of quantum mechanics demand no cloning and no destruction of qubits. This means that as we map the number of classical bits to qubits, for the classical routine to be directly convertible to quantum we must also ensure that all bits in equals the number of bits out. Unitarity also demands a unique combination of output qubits per combination of input qubits. This all is the equivalent of saying that the routines must be reversible. Finding

reversible computing routines that can be converted to quantum computing allows us to design these quantum versions of classical routines, for example basic mathematical operations like addition and multiplication.

While those addition and multiplication routines are not expected to themselves offer speedup or any kind of quantum advantage over classical routines, by integrating them with such algorithms that do, users can keep their software implementations streamlined and avoid the major bottleneck of moving data between a QPU and a CPU (Cruise et al., 2020).

This work has been explored before on other platforms. For example, Fedoriaka et al. (2025) explored each of our implementations in Microsoft's Q# platform. Therefore we have some baseline for benchmarking and implementing these adders and multipliers. However, the algorithms in that paper have largely not implemented in our target platform, Workbench by PsiQuantum. The platform itself is implemented to be abstracted from the hardware and so our implementation will not directly assume a particular hardware implementation. Workbench uses C++ optimized simulators supporting both superposition for full quantum effects or (relevant to our case) non-superposition qubit registers which allow for many more simulated qubits to be held in memory simultaneously. The latter is useful for our use case as arithmetic operations don't utilize superposition states and are designed for direct quantum analogs of classical bit registers.

The three types of adders we have implemented already fall into two different categories, as do all adders. In-place adders take two registers of N qubits and 1 carry qubit and store the result in one of the N qubit registers. In that sense, they are constant in memory usage. However, this comes at the cost of complexity of algorithm implementation, which manifests as circuit depth in quantum programming. The alternative is to use out-of-place adders, which uses an additional auxiliary register also of N qubits. These use the additional memory space to reduce the complexity of the algorithm and the quantum circuit depth, with the trade-off of extra memory usage. Of the three algorithms we use, TTK (Takahashi et al., 2009) is an example of an in-place adder, and CT (Cheng & Tseng, 2002) and Wang (Wang et al., 2016) are out-of-place adders. Existing implementations in Workbench are in-place adders Gidney (Gidney, C., 2018), CDKM (Cuccaro et al., 2004), and QFT (Draper, T., 2000), the latter of which uses a quantum Fourier transform to perform addition. Most of these are "ripple-carry" adders, which simply means that, as the addition per qubit is modulo 2, the carry propagates from the little-endian side of the register to the big-endian side as if it's rippling.

2. Methods

We have developed reversible computing routines for three algorithms: TTK, CT, and Wang. TTK is designed to utilize no additional qubits at the tradeoff of higher complexity (Wang et al., 2016). CT is an older algorithm that scales 1-qubit addition to N -qubit addition (Cheng, 2002), mainly serving as a reference point for the field. Wang is a more recent algorithm designed specifically for minimizing complexity of the quantum circuit in exchange for increasing the amount of qubits used in memory (Wang et al., 2016).

We have implemented these algorithms in Workbench, the Python library implemented in PsiQuantum's software development, circuit design, and resource analyzing tool Construct. Workbench is designed to allow writing of quantum programs in a familiar, Pythonic manner, e.g. accessing qubits in a register by slicing through the register like a Python list or NumPy array. This allows more rapid implementation, iteration, and testing of a prospective quantum program or routine, and less room for design or implementation error from misunderstanding the API. One instantiates a QPU object with an allocation of qubits, as well as specifying whether to use a simulation that handles superposition in exchange for fewer qubits or vice

versa. One then calls quantum operations which map to several of the most common gates used in quantum information, e.g. the X , Y , and Z gates, as well as CNOT, Toffoli/CCNOT, and rotation gates. We did not directly use rotation gates in our algorithms as they were not needed; however, Toffoli gates are implemented with a number of T gates (Jones, 2013) which are expensive rotation gates (Orts et al., 2023), even if the coder isn't exposed to it.

High-level Workbench library routines are implemented as “qubricks.” These are essentially analogs to Python classes designed to accommodate quantum information processes. The syntax is for the most part exactly that of a Python class, with the requirement of inheriting the Qubrick class. There are several more requirements to a qubrick instead of a regular Python class. There is always a specific routine that is the expected entry point of the qubrick. The qubricks aren't allowed to return values, and there are prescribed methods to handle function outputs of a qubrick like with arithmetic. Auxiliary qubits assigned internally are managed by the qubrick. At all steps, operations that correspond to destructive measurements of qubit registers must be avoided, as presumably the user did not yet intend for the superposition to collapse; everything must be handled by unitary gates and prescribed API routines to avoid this.

Finally, besides wave function collapse, care must be taken to ensure that the qubit registers sent in by the user are the same ones they get back unless deliberately intended otherwise. An example of when they might be intended to be altered is in the case of incremental addition i.e. $a += b$. Otherwise, for addition inputs a and b , they are not to be altered, nor is b in the case of incremental addition.

This is non-trivial, as some routines e.g. Wang specifically work in a conceptual manner by “shuffling around” qubits between the input registers and the additional auxiliary register (Wang is an “out-of-place” adder), so that in general a stores the sum result but b stores some new integer which in general is not the same as what the user expected when passing it in. This can be handled in Workbench by using a prescribed routine to temporarily store the value of a register, in this case a , then by “un-computing,” which is parlance for performing all the steps in the algorithm exactly in reverse. While this doubles runtime, it is a necessary feature to ensure correctness and reliability, such as the getting back a and b as they were passed in. The qubrick's documentation does need to take care to specify its signature for usage such as whether or not addition is incremental or is sent back to a new variable.

There is an additional reason to “uncompute” the results, particular in the case of auxiliary qubits interior to the qubrick. It is quite easy for any such internal qubits to become entangled to the input registers in a way that creates non-trivial errors. We heard the example of a qubrick trying to set one of the input registers back to zero without the entangled auxiliary being uncomputed. This induced a relative phase which caused a two qubit state to become the wrong superposition state, such that a Hadamard operator then sent the input qubit back to a 1 state instead of a 0 state as intended. These errors are subtle and if not deliberately checked for can crop up in insidiously unexpected ways down the line. They are deeply interrelated to the problem of checking that input variables have not spontaneously changed due to the algorithm's main computation.

Thankfully, Workbench was designed with these problems in mind, and there are built-in routines that one can call to handle this automatically, along with configurable options (our algorithms generally used the simplest, default configuration). Workbench also works with Python's “with” resource-management syntax to handle computation, retrieving the result, and uncomputation all at once. As long as we were aware of this, and designed our exact code implementations accordingly, this was easy enough to accommodate. Furthermore, as part of a continuous integration testing suite for our git repo, we had unit tests checking for

consistency of input registers, with a switch for in-place versus out-of-place adders, so that this could be monitored automatically.

Benchmarking algorithms is an important component of the project. Workbench naturally supports this with built-in functions for resource estimation of a given quantum circuit, both in terms of a Python dictionary (hash map) that contains the set of quantum gates with nonzero instances in the circuit and the number of instances associated with them. We are primarily interested in following the number of T gates as they are the most expensive (Orts et al., 2023), along with general considerations of circuit depth and memory usage. There is also a quick and easy diagram drawn from a one-line callable routine, which is very useful. It is quite easy to set up our desired analytics, and we show some preliminary results in section 3. This will also allow us to attempt further optimizations down the line.

The adders take in Workbench data types Qubits and QUInt which functions as a hybrid Qubits object and an unsigned integer. Results are not returned directly but are returned either by setting a classical or quantum register to hold the result via built-in routines.

3. Results and Analysis

The reference for performance for our work is in Fedoriaka, et al. (2025) with their Q# implementation. We perform benchmarking with the Workbench resource estimator. The baseline theory expectations from their respective papers are, for registers of N qubits:

	CT	TTK	Wang
Gates Required	$9N$	$7N - 6$	$6N - 2$
Toffoli Gates Required	$4N$	$2N - 1$	N
Qubits Required	$4N + 1$	$2N + 1$	$3N + 1$

Toffoli gates are decomposed in part into expensive T gates. The canonical implementation in quantum circuits literature (Orts et al., 2023) is by Amy et al. (2013) with seven T gates. So these involve $28N$, $14N - 7$, and 7 T gates respectively. Our preliminary results are:

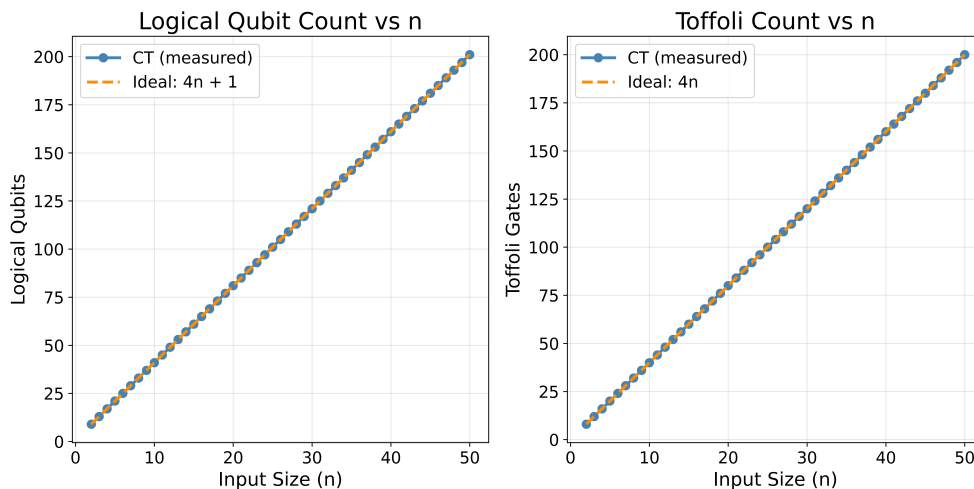


Figure 1: Preliminary benchmarks for the CT adder

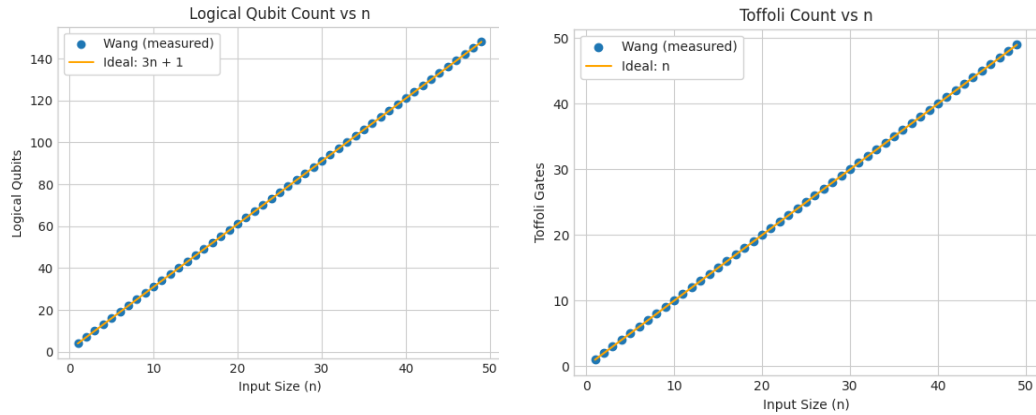


Figure 2: Preliminary benchmarks for the Wang adder

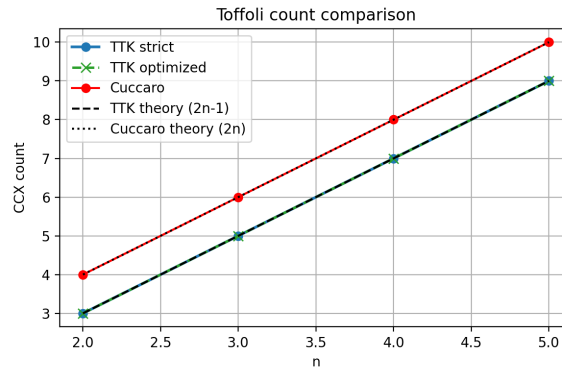


Figure 3: Preliminary Toffoli benchmark for the TTK adder (qubits not available)

These are preliminary analytics. We would eventually like to track physical qubits and wall clock-time as Workbench facilitates it to match the benchmarks in Fedoriaka et al. (2025). We also need to standardize plotting across all algorithms as part of our pipeline.

It should be noted that constant offsets are generally not very worrisome, for example the difference between TTK and Cuccaro in Figure 3. The leading-order term typically dominates scaling with N qubits for analysis of algorithm complexity in time and space usage.

4. Future Work

We will implement more algorithms essentially in the same way as the ones we've covered. In particular, we would like to implement at least two of each class of algorithm: in-place adder, out-of-place adder, and multiplier to get like-type comparisons in the benchmarking. The current goal is to do six total, with another in-place adder and two multipliers left to do. We will explore optimizations to be made and possible additional algorithms to test; the reference paper of Fedoriaka et al. (2025) tested seven in-place adders, seven out-of-place adders, and six multipliers, in addition to two dividers. We will not likely reproduce that in full but if time permits we would like to do as much as we can. We will also finish standardizing our plotting pipeline.

The main risks are pushing too hard on implementing algorithms such that we are not integrating them properly into Workbench or testing them properly. To mitigate this, our continuous integration is mostly there but needs to be completely finished before moving much further. We have also seen service outages affecting our ability to use the cloud environment behind Workbench in its current state, which causes delays (in particular, in finalizing plots for this report). Mitigating this requires more proactive and early work as well as active communication of issues with our industry mentor so that they may be relayed to the internal team for diagnosis.

5. Project Schedule

The original and updated Gantt charts are provided. A single entry for implementing second algorithms has been added in lieu of duplicating all previous entries, but it should be understood that all steps are required for all algorithms.



Figure 4: Original Gantt chart for preliminary project timeline for reference

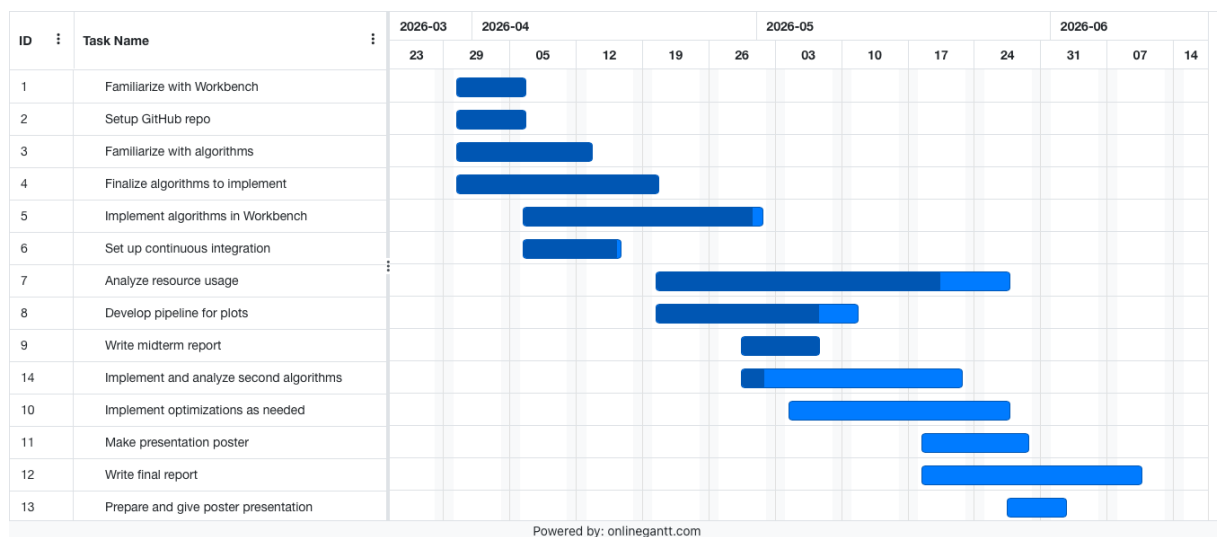


Figure 5: Updated Gantt chart with project timeline; completion status filled in with dark blue

The previous schedule was largely accurate except that too much time was set up for analyzing resource usage which turns out to be relatively simple. A plots pipeline has also been relatively straightforward to implement except for the marginal problem of standardization. Continuous integration has proven harder and getting so far has taken up disproportionate time; at the same time, it is foundational and so needed to be done earlier. As mentioned, it will need to be completely finished as soon as possible.

Team members remain responsible for implementing their own algorithms in Workbench and for testing and analyzing them as they push to the git repo.

AI Use Disclosure

AI has been used for debugging, polishing, and writing small amounts of code used in the project. No AI was used for writing this paper.

References

- Amy, M., et al. "A Meet-in-the-Middle Algorithm for Fast Synthesis of Depth-Optimal Quantum Circuits." IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 32, no. 6, pp. 818-830, June 2013, doi: 10.1109/TCAD.2013.2244643.
- Babu, H. M. H. Quantum Computing (Second Edition), Ch 4. IOP Publishing. June 2023.
- Cheng, K. W. and Tseng, C. C. "Quantum full adder and subtractor." November 2002. Electronics Letters. 38(22):1343 - 1344.
- Cruise, J. R.; Gillespie, N., I.; and Reid, B. "Practical Quantum Computing: The value of local computation." 21 September 2020.
- Cuccaro, S., et al. "A new quantum ripple-carry addition circuit." 22 October 2004.
- Draper, T. G. "Addition on a Quantum Computer." 7 August 2000.
- Fedorika, D.; Goldsmith, B.; and Chen, Y. "Quantum Arithmetic Algorithms: Implementation, Resource Estimation, and Comparison." 6 September 2025.
- Jones, C. "Low-overhead constructions for the fault-tolerant Toffoli gate." 20 February 2013. Phys. Rev. A 87, 022328.
- Orts, F., et al. "Improving the number of T gates and their spread in integer multipliers on quantum computing." Physical Review A, vol. 107, no. 4, p. 042621, 2023.
- Takahashi, Y.; Tani, S.; and Kunihiro, N.. "Quantum Addition Circuits and Unbounded Fan-Out." 14 October 2009.
- Wang, F., Luo, M., Li, H. et al. "Improved quantum ripple-carry addition circuit." 2016. Sci. China Inf. Sci. 59, 042406.